

PINN-MLP vs PINN-KAN: Análisis Completo del Código

Ecuación de Poisson 1D resuelta con Redes Neuronales Informadas por Física

1. El Problema Físico: Ecuación de Poisson 1D

El código resuelve la siguiente ecuación diferencial parcial (EDP):

$$\begin{aligned} u_{xx}(x) &= -\sin(x), & x \in [0, \pi] \\ u(0) &= 0, & u(\pi) = 0 \quad (\text{condiciones de contorno}) \end{aligned}$$

donde u_{xx} denota la segunda derivada de u respecto a x . La solución analítica exacta es $u(x) = \sin(x)$, que satisface la EDP y las condiciones de contorno.

¿Por qué este problema?

La ecuación de Poisson es el "Hola Mundo" de las EDPs. Aparece en electrostática, transferencia de calor y mecánica de fluidos. Tener solución exacta ($\sin(x)$) nos permite verificar el error del modelo neural con precisión.

2. ¿Qué es una PINN? (Physics-Informed Neural Network)

Una PINN es una red neuronal cuya función de pérdida incluye no solo datos de entrenamiento, sino también el residuo de una ecuación diferencial. La idea es que la red aprenda una solución que satisfaga simultáneamente:

Condición 1 — La EDP: El residuo $R(x) = u_{xx}(x) - f(x)$ debe ser cero en puntos del dominio.

Condición 2 — Las condiciones de contorno (CC): La solución debe valer 0 en $x=0$ y $x=\pi$.

La función de pérdida total es:

$$\begin{aligned} L &= L_{\text{PDE}} + L_{\text{CC}} \\ L_{\text{PDE}} &= (1/N) * \sum_i [u_{xx}(x_i) - f(x_i)]^2 \\ L_{\text{CC}} &= u(0)^2 + u(\pi)^2 \end{aligned}$$

Minimizar esta pérdida mediante descenso de gradiente obliga a la red a encontrar una función que se comporte como la solución de la EDP. No se necesitan datos de la solución real — solo los collocation points (puntos del dominio donde evaluamos el residuo).

3. Configuración del Dominio y Parámetros

```
N_points = 100
x = np.linspace(0, np.pi, N_points).reshape(-1, 1)
f_x = -np.sin(x)
u_exact = np.sin(x)
epochs = 1000
```

N_points = 100: Se discretiza el intervalo $[0, \pi]$ en 100 puntos igualmente espaciados.
x = np.linspace(...).reshape(-1,1): Genera el vector de colocation points. El reshape convierte el array de forma (100,) a forma (100,1), necesario para álgebra matricial.
f_x = -sin(x): El término fuente de la EDP. Forma (100,1).
u_exact = sin(x): La solución exacta, usada solo para visualización final.
epochs = 1000: Número de iteraciones del descenso de gradiente.

4. PINN con MLP — Red Neuronal Estándar

4.1 Arquitectura de la MLP

La red tiene una única capa oculta con 20 neuronas y función de activación tanh.
 Esquemáticamente:

$$x \longrightarrow [W, b] \longrightarrow z = x \cdot W + b \longrightarrow a = \tanh(z) \longrightarrow [V] \longrightarrow u = a \cdot V$$

Los parámetros son:

- $W \in \mathbb{R}^{(1 \times H)}$: pesos de entrada (conexiones de x a cada neurona)
- $b \in \mathbb{R}^{(1 \times H)}$: sesgos de cada neurona
- $V \in \mathbb{R}^{(H \times 1)}$: pesos de salida (conexiones de neuronas a la salida)

```
H = 20
np.random.seed(42)
W = np.random.randn(1, H) * 0.1 # forma (1, 20)
b = np.zeros((1, H))           # forma (1, 20)
V = np.random.randn(H, 1) * 0.1 # forma (20, 1)
lr_mlp = 0.005
```

H = 20: Número de neuronas en la capa oculta.

np.random.seed(42): Semilla para reproducibilidad.

W y V * 0.1: Inicialización pequeña para evitar saturación de tanh al inicio del entrenamiento. Si los pesos fuesen grandes, $\tanh(z) \approx \pm 1$ desde el principio y los gradientes serían casi cero.

b = zeros: Inicialización a cero estándar para sesgos.

4.2 Forward Pass y Cálculo de u_xx

En cada epoch, primero se calcula la predicción de u y su segunda derivada de forma completamente analítica:

```
z = np.dot(x, W) + b # (100,1) · (1,20) + (1,20) = (100,20)
a = np.tanh(z)       # (100,20) - activaciones
u_pred = np.dot(a, V) # (100,20) · (20,1) = (100,1)
```

z = x·W + b: La preactivación. Cada punto x_i produce un vector de 20 valores $z_i \in \mathbb{R}^{20}$.

a = tanh(z): La activación. Aplica tanh elemento a elemento.

u_pred = a·V: La predicción de la solución: $u(x) = \sum_{j=1}^{20} V_j \cdot \tanh(W_j \cdot x + b_j)$

Ahora viene la parte clave de una PINN: necesitamos evaluar la EDP, lo que requiere calcular la segunda derivada de u respecto a x . En lugar de usar diferenciación automática, el código lo hace analíticamente:

Primera derivada de u respecto a x :

$$u_x = \sum_j V_j \cdot \tanh'(z_j) \cdot W_j$$

$$\tanh'(z) = \text{sech}^2(z) = 1 - \tanh^2(z)$$

$$\therefore u_x = \sum_j V_j \cdot (1 - a_j^2) \cdot W_j$$

Segunda derivada de u respecto a x :

$$u_{xx} = \sum_j V_j \cdot d/dx[\text{sech}^2(z_j)] \cdot W_j$$

$$d/dx[\text{sech}^2(z_j)] = d/dz[\text{sech}^2(z)] \cdot dz/dx = -2 \cdot \tanh(z) \cdot \text{sech}^2(z) \cdot W_j$$

$$\therefore u_{xx} = \sum_j V_j \cdot (-2 \cdot a_j \cdot \text{sech}^2(z_j)) \cdot W_j^2$$

```
a2 = a**2                                # (100,20) - tanh^2(z)
sech2 = 1 - a2                           # (100,20) - sech^2(z) = 1-tanh^2(z)
u_xx = np.dot(-2 * a * sech2 * (W**2), V) # (100,1)
```

La expresión $-2 \cdot a \cdot \text{sech}^2 \cdot (W^2)$ tiene forma (100,20) — cada elemento (i,j) es $-2 \cdot a_{ij} \cdot \text{sech}^2_{ij} \cdot W_{ij}^2$. Al multiplicar por V de forma (20,1), la suma sobre j da la segunda derivada para cada punto x_i .

4.3 Pérdida

```
R = u_xx - f_x                        # Residuo PDE, forma (100,1)
L_pde = np.mean(R**2)                 # MSE del residuo
L_bc = u_pred[0,0]**2 + u_pred[-1,0]**2 # CC: u(0)^2+u(pi)^2
loss_history_mlp.append(L_pde + L_bc)
```

Residuo R: Es la cantidad que queremos minimizar. Si $R=0$ en todos los puntos, la EDP está satisfecha exactamente.

L_{pde} : MSE del residuo — penaliza desviaciones de la EDP.

L_{bc} : Penaliza que la red no valga 0 en los extremos. Se usan los índices [0,0] ($x=0$) y [-1,0] ($x=\pi$).

4.4 Gradientes Analíticos — PDE

Para actualizar los parámetros, necesitamos calcular $\partial L / \partial W$, $\partial L / \partial b$, $\partial L / \partial V$. Usando la regla de la cadena:

$$\partial L_{PDE} / \partial \theta = (2/N) \cdot \sum_i R_i \cdot \partial u_{xx}(x_i) / \partial \theta$$

Gradiente respecto a V :

$$\partial u_{xx} / \partial V_j = -2 \cdot a_j \cdot \text{sech}^2(z_j) \cdot W_j^2$$

```
d_uxx_dV = -2 * a * sech2 * (W**2)      # (100,20)
grad_V = np.mean(2 * R * d_uxx_dV, axis=0, keepdims=True).T # (20,1)
```

Gradiente respecto a b :

Necesitamos $\partial u_{xx} / \partial b_j$. Derivando la fórmula de u_{xx} respecto a b_j (notando que $\partial z_j / \partial b_j = 1$):

$$\partial / \partial b_j [-2 \cdot a_j \cdot \text{sech}^2(z_j) \cdot W_j^2] = -2 \cdot W_j^2 \cdot V_j \cdot \text{sech}^2(z_j) \cdot (1 - 3 \cdot a_j^2)$$

Demostración: sea $g(z) = -2 \cdot \tanh(z) \cdot \text{sech}^2(z)$. Entonces:

```

g'(z) = -2[sech2(z)2 + tanh(z) · (-2 · sech2(z) · tanh(z))]
       = -2 · sech2(z) [sech2(z) - 2 · tanh2(z)]
       = -2 · sech2(z) [(1-a2) - 2a2]
       = -2 · sech2(z) · (1 - 3a2)

```

```

d_uxx_db = -2 * V.T * (W**2) * (1 - 3*a2) * sech2 # (100,20)
grad_b = np.mean(2 * R * d_uxx_db, axis=0, keepdims=True) # (1,20)

```

Gradiente respecto a W:

Ahora W aparece dos veces en $u_{xx} = \sum_j V_j \cdot (-2 \cdot a_j \cdot \text{sech}^2(z_j)) \cdot W_j^2$, tanto en W_j^2 explícito como en $a_j = \tanh(W_j \cdot x + b_j)$. Aplicando la regla del producto:

```

∂/∂W_j [V_j · (-2 · a_j · sech2(z_j)) · W_j2]
= V_j · [(-2 · a_j · sech2(z_j)) · 2W_j + W_j2 · g'(z_j) · x]
= -2 · V_j · sech2(z_j) · [2 · a_j · W_j + W_j2 · x · (1 - 3a_j2)]

```

```

d_uxx_dW = -2 * V.T * (2*W*a*sech2 + (W**2)*x*(1-3*a2)*sech2) # (100,20)
grad_W = np.mean(2 * R * d_uxx_dW, axis=0, keepdims=True) # (1,20)

```

4.5 Gradientes de las Condiciones de Contorno

La pérdida de CC es $L_{CC} = u(0)^2 + u(\pi)^2$. Sus gradientes son:

```

∂LCC/∂V_j = 2 · u(0) · a_j(x=0) + 2 · u(π) · a_j(x=π)
∂LCC/∂b_j = 2 · u(0) · V_j · sech2(z_j(0)) + 2 · u(π) · V_j · sech2(z_j(π))
∂LCC/∂W_j = ∂LCC/∂b_j · x[0] o x[-1] (escalar de x)

```

```

d_u_dV_0, d_u_dV_pi = a[0:1,:].T, a[-1:,:].T # (20,1) cada uno
d_u_db_0, d_u_db_pi = V.T*sech2[0:1,:], V.T*sech2[-1:,:] # (1,20)
d_u_dW_0, d_u_dW_pi = d_u_db_0*x[0], d_u_db_pi*x[-1] # (1,20)

```

```

grad_V += 2*u_pred[0,0]*d_u_dV_0 + 2*u_pred[-1,0]*d_u_dV_pi
grad_b += 2*u_pred[0,0]*d_u_db_0 + 2*u_pred[-1,0]*d_u_db_pi
grad_W += 2*u_pred[0,0]*d_u_dW_0 + 2*u_pred[-1,0]*d_u_dW_pi

```

4.6 Actualización de Parámetros

```

W -= lr_mlp * grad_W
b -= lr_mlp * grad_b
V -= lr_mlp * grad_V

```

Descenso de gradiente estándar (SGD) con tasa de aprendizaje 0.005.

5. KAN — Kolmogorov-Arnold Networks: Concepto Global

5.1 Motivación: ¿Qué falla con las MLP?

Las MLP aplican funciones de activación fijas (tanh, ReLU) a los nodos. La red aprende los pesos, pero la no-linealidad siempre tiene la misma forma. Esto es una restricción arquitectónica.

Intuición clave

En una MLP: activaciones fijas en nodos, pesos aprendibles en aristas.
 En una KAN: activaciones aprendibles en aristas, nodos solo suman.

5.2 El Teorema de Kolmogorov-Arnold

El teorema establece que cualquier función continua multivariable puede descomponerse como composición de funciones univariables:

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \varphi_{q,p}(x_p) \right)$$

donde cada $\varphi_{q,p}$ y Φ_q son funciones univariables aprendibles. Esto justifica arquitecturas donde las aristas transportan transformaciones no lineales aprendidas en lugar de simples multiplicaciones de peso.

5.3 Arquitectura KAN: Funciones en Aristas

En una KAN, cada conexión entre nodos aplica una función no lineal aprendible $\varphi(x)$. En la implementación original (Liu et al. 2024), se usan B-splines. En este código, se usa una aproximación con funciones gaussianas:

$$\varphi_j(x) = C_j \cdot \exp(-0.5 \cdot ((x - \mu_j)/\sigma_j)^2)$$

Esto es en realidad una Red de Funciones de Base Radial Gaussiana (GRBF), que captura el espíritu KAN: las aristas tienen activaciones con forma aprendible mediante los parámetros (C, μ, σ) .

La salida de la red KAN es simplemente la suma de todas las contribuciones:

$$u(x) = \sum_{j=1}^H C_j \cdot \varphi_j(x) = \sum_{j=1}^H C_j \cdot \exp(-0.5 \cdot ((x - \mu_j)/\sigma_j)^2)$$

Tabla comparativa MLP vs KAN

MLP: $u(x) = \sum_j V_j \cdot \tanh(W_j \cdot x + b_j) \rightarrow$ forma fija, escala/desplaza
KAN: $u(x) = \sum_j C_j \cdot \exp(-0.5 \cdot ((x - \mu_j)/\sigma_j)^2) \rightarrow$ forma adaptable por μ, σ

En la KAN, μ controla DÓNDE está el centro de la activación (posición en x).

σ controla CUÁNTO se extiende la activación (ancho de la campana).

C escala la amplitud de la contribución de esa neurona.

5.4 Segunda Derivada de $\varphi_j(x)$

Para la PINN necesitamos u_{xx} . Derivemos φ_j analíticamente. Sea:

$$z_j = (x - \mu_j)/\sigma_j, \quad \varphi_j = \exp(-0.5 \cdot z_j^2)$$

Primera derivada:

$$\varphi_j'(x) = \exp(-0.5 \cdot z_j^2) \cdot (-z_j) \cdot (1/\sigma_j) = -\varphi_j \cdot z_j / \sigma_j$$

Segunda derivada:

$$\begin{aligned} \varphi_j''(x) &= d/dx [-\varphi_j \cdot z_j / \sigma_j] \\ &= -(\varphi_j' \cdot z_j + \varphi_j \cdot (1/\sigma_j)) / \sigma_j \\ &= -(-\varphi_j \cdot z_j / \sigma_j \cdot z_j + \varphi_j / \sigma_j) / \sigma_j \\ &= \varphi_j \cdot (z_j^2 - 1) / \sigma_j^2 \end{aligned}$$

Y entonces: $u_{xx} = \sum_j C_j \cdot \varphi_j'' = \sum_j C_j \cdot \varphi_j \cdot (z_j^2 - 1) / \sigma_j^2$

6. PINN con KAN — Código Línea por Línea

6.1 Inicialización

```
C = np.zeros((H, 1)) # Coeficientes, forma (20,1)
mu = np.linspace(0, np.pi, H).reshape(1, H) # Centros, forma (1,20)
sigma = np.ones((1, H)) * (np.pi/H) * 1.5 # Anchuras, forma (1,20)

lr_C = 0.005
lr_mu = 0.005
lr_sigma = 0.005
```

C = zeros: Inicialmente todos los coeficientes son cero. La red no predice nada al inicio.

mu = linspace(0,π,H): Los centros se distribuyen uniformemente por el dominio. Esto asegura cobertura completa de $[0,\pi]$.

sigma = (π/H)·1.5: Anchura inicial = separación entre centros × 1.5. Garantiza solapamiento entre gaussianas adyacentes para cobertura continua.

6.2 Forward Pass KAN

```
z = (x - mu) / sigma # (100,1) - (1,20) / (1,20) = (100,20)
phi = np.exp(-0.5 * z**2) # (100,20) - gaussianas evaluadas
u_pred_kan = np.dot(phi, C) # (100,20) · (20,1) = (100,1)

z2 = z**2
phi_xx = phi * (z2 - 1) / (sigma**2) # (100,20) - segunda derivada de φ
u_xx_kan = np.dot(phi_xx, C) # (100,1)
```

z = (x - mu)/sigma: Broadcasting de numpy: x tiene forma (100,1), mu y sigma tienen forma (1,20). El resultado es (100,20): cada fila i contiene los 20 valores $z_{\{i,j\}} = (x_i - \mu_j)/\sigma_j$.

phi: Las 20 gaussianas evaluadas en los 100 puntos. $\phi_{\{i,j\}} = \exp(-0.5 \cdot z_{\{ij\}}^2)$.

u_pred_kan: La predicción: $u(x_i) = \sum_j C_j \cdot \phi_j(x_i)$.

phi_xx: La segunda derivada: $\phi_{xx}[i,j] = \phi_j''(x_i) = \phi_j(x_i) \cdot (z_{\{ij\}}^2 - 1)/\sigma_j^2$.

u_xx_kan: La segunda derivada de u.

6.3 Gradientes KAN — PDE

Gradiente respecto a C:

$$\partial u_{xx} / \partial C_j = \phi_j''(x) \rightarrow d_{u_{xx}} dC = \phi_{xx} \quad (100,20)$$

Gradiente respecto a μ_j :

Necesitamos $\partial u_{xx} / \partial \mu_j$. Partiendo de $u_{xx} = \sum_k C_k \cdot \phi_k \cdot (z_k^2 - 1) / \sigma_k^2$, solo el término $k=j$ depende de μ_j :

$$\partial / \partial \mu_j [\phi_j \cdot (z_j^2 - 1) / \sigma_j^2]$$

Nota: $\partial z_j / \partial \mu_j = -1/\sigma_j$, $\partial \phi_j / \partial \mu_j = \phi_j \cdot (z_j / \sigma_j)$
 $= (\phi_j \cdot z_j / \sigma_j) \cdot (z_j^2 - 1) / \sigma_j^2 + \phi_j \cdot (2z_j \cdot (-1/\sigma_j)) / \sigma_j^2$
 $= \phi_j / \sigma_j^3 \cdot [z_j(z_j^2 - 1) - 2z_j]$
 $= \phi_j \cdot z_j \cdot (z_j^2 - 3) / \sigma_j^3$
 $= \phi_j \cdot (z_j^3 - 3z_j) / \sigma_j^3$

```
d_u_xx_dmu = C.T * phi * (z**3 - 3*z) / (sigma**3) # (100,20)
```

Gradiente respecto a σ_j :

Análogamente, $\partial z_j / \partial \sigma_j = -z_j / \sigma_j$, $\partial \varphi_j / \partial \sigma_j = \varphi_j \cdot (z_j^2 / \sigma_j)$:

$$\begin{aligned} \partial / \partial \sigma_j [\varphi_j \cdot (z_j^2 - 1) / \sigma_j^2] \\ &= (\varphi_j \cdot z_j^2 / \sigma_j) \cdot (z_j^2 - 1) / \sigma_j^2 + \varphi_j \cdot [2z_j \cdot (-z_j / \sigma_j)] / \sigma_j^2 + \varphi_j \cdot (z_j^2 - 1) \cdot (-2 / \sigma_j^3) \\ &= \varphi_j / \sigma_j^3 \cdot [z_j^2 (z_j^2 - 1) - 2z_j^2 - 2(z_j^2 - 1)] \\ &= \varphi_j \cdot (z_j^4 - 5z_j^2 + 2) / \sigma_j^3 \end{aligned}$$

```
d_uxx_dsigma = C.T * phi * (z**4 - 5*z2 + 2) / (sigma**3) # (100,20)
```

6.4 Gradientes KAN — Condiciones de Contorno

Para $u(x) = \sum_j C_j \cdot \varphi_j(x)$:

$$\partial u / \partial C_j = \varphi_j(x)$$

$$\partial u / \partial \mu_j = C_j \cdot \varphi_j(x) \cdot (z_j(x) / \sigma_j)$$

$$\partial u / \partial \sigma_j = C_j \cdot \varphi_j(x) \cdot (z_j(x)^2 / \sigma_j)$$

```
d_u_dC_0, d_u_dC_pi = phi[0:1,:].T, phi[-1,:].T # (20,1)
d_u_dmu_0, d_u_dmu_pi = C.T*phi[0:1,:]*(z[0:1,:]/sigma), C.T*phi[-1,:]*(z[-1,:]/sigma) # (1,20)
d_u_dsig_0, d_u_dsig_pi = C.T*phi[0:1,:]*(z2[0:1,:]/sigma), C.T*phi[-1,:]*(z2[-1,:]/sigma) # (1,20)
```

```
grad_C += 2*u_pred_kan[0,0]*d_u_dC_0 + 2*u_pred_kan[-1,0]*d_u_dC_pi
grad_mu += 2*u_pred_kan[0,0]*d_u_dmu_0 + 2*u_pred_kan[-1,0]*d_u_dmu_pi
grad_sigma += 2*u_pred_kan[0,0]*d_u_dsig_0 + 2*u_pred_kan[-1,0]*d_u_dsig_pi
```

6.5 Actualización

```
C -= lr_C * grad_C
mu -= lr_mu * grad_mu
sigma -= lr_sigma * grad_sigma
```

Descenso de gradiente con tasas desacopladas (aunque en este código tienen el mismo valor de 0.005).

7. Análisis Exhaustivo del Código: Corrección y Errores

7.1 Verificación de Dimensiones — MLP

Se verifican todas las formas para asegurar consistencia del álgebra matricial:

```
x: (100, 1)
W: (1, 20) → z = x·W + b: (100,20) ✓
a, sech2: (100,20)
V: (20, 1) → u_pred = a·V: (100,1) ✓
u_xx: (100, 1) ← dot(-2*a*sech2*(W²), V): (100,20) · (20,1) ✓
R: (100, 1)
d_uxx_dV: (100,20) → 2*R*d_uxx_dV: (100,20), mean→(1,20).T=(20,1)=V ✓
d_uxx_db: (100,20) → 2*R*d_uxx_db: (100,20), mean→(1,20)=b ✓
d_uxx_dW: (100,20) → 2*R*d_uxx_dW: (100,20), mean→(1,20)=W ✓
```

✓ Dimensiones MLP verificadas correctamente

Todos los tensores tienen las formas correctas para el álgebra matricial. No hay errores de dimensionalidad.

7.2 Verificación de Dimensiones — KAN

```
x:      (100, 1)
mu:      ( 1, 20)
sigma:   ( 1, 20)
z:      (100, 20) ← broadcasting (100,1)-(1,20)/(1,20) ✓
phi:     (100, 20)
C:       ( 20, 1)
u_pred:  (100, 1) ← phi·C ✓
phi_xx:  (100, 20)
u_xx_kan: (100, 1) ← phi_xx·C ✓
d_uxx_dC: (100, 20) = phi_xx → grad_C=(20,1) ✓
C.T:     ( 1, 20)
d_uxx_dmu: (100, 20) → grad_mu=(1,20) ✓
d_uxx_dsigma: (100,20)→ grad_sigma=(1,20) ✓
```

✓ Dimensiones KAN verificadas correctamente

El broadcasting de numpy funciona correctamente. Todas las operaciones matriciales tienen dimensiones compatibles.

7.3 Verificación de las Fórmulas de Gradiente — MLP

$u_{xx} = \text{dot}(-2 \cdot a \cdot \text{sech}^2 \cdot W^2, V)$: ✓ Correcto. La derivada de $\tanh(Wx+b)$ dos veces da exactamente $-2 \cdot \tanh \cdot \text{sech}^2 \cdot W^2$.

$d_{uxx}_{db} = -2 \cdot V.T \cdot W^2 \cdot (1-3a^2) \cdot \text{sech}^2$: ✓ Correcto. La derivada de $g(z) = -2 \tanh \cdot \text{sech}^2$ es $g'(z) = -2 \cdot \text{sech}^2 \cdot (1-3 \tanh^2)$.

$d_{uxx}_{dW} = -2 \cdot V.T \cdot (2W \cdot a \cdot \text{sech}^2 + W^2 \cdot x \cdot (1-3a^2) \cdot \text{sech}^2)$: ✓ Correcto. Regla del producto aplicada correctamente a la doble dependencia de W .

7.4 Verificación de las Fórmulas de Gradiente — KAN

$\phi_{xx} = \phi \cdot (z^2-1)/\sigma^2$: ✓ Correcto. Segunda derivada de $\exp(-z^2/2)$ analíticamente verificada.

$d_{uxx}_{dmu} = C \cdot \phi \cdot (z^3-3z)/\sigma^3$: ✓ Correcto. Equivalente a $C \cdot \phi \cdot z \cdot (z^2-3)/\sigma^3$.

$d_{uxx}_{dsigma} = C \cdot \phi \cdot (z^4-5z^2+2)/\sigma^3$: ✓ Correcto. Derivada polinómica verificada paso a paso.

$d_{u}_{dmu} = C \cdot \phi \cdot (z/\sigma)$: ✓ Correcto. $\partial \phi / \partial \mu = \phi \cdot z / \sigma$.

$d_{u}_{dsig} = C \cdot \phi \cdot (z^2/\sigma)$: ✓ Correcto. $\partial \phi / \partial \sigma = \phi \cdot z^2 / \sigma$.

✓ Todos los gradientes son matemáticamente correctos

Los gradientes analíticos de la MLP y la KAN han sido verificados paso a paso con cálculo diferencial. No hay errores matemáticos en ninguno.

7.5 Problemas y Advertencias Identificados

⚠ ADVERTENCIA 1: Nomenclatura — Esto no es una KAN "pura"

El código llama KAN a lo que realmente es una Red de Funciones de Base Radial (RBF). Una KAN verdadera (Liu et al., 2024) usa B-splines como funciones de arista y tiene una arquitectura multicapa donde cada capa computa sumas de funciones aprendibles. Esta implementación es una capa de gaussianas aprendibles — tiene el espíritu de KAN (activaciones aprendibles en aristas) pero no la arquitectura completa.

⚠ ADVERTENCIA 2: C inicializado a cero

Con $C=0$ al inicio, la red KAN no predice nada ($u=0$). Esto es técnicamente válido, pero puede hacer que algunos gradientes iniciales respecto a μ y σ sean también cero (ya que d_{uxx_dmu} y d_{uxx_dsigma} se multiplican por C). Los centros y anchos podrían no moverse al principio. Inicializar C con valores pequeños aleatorios mejoraría la dinámica de entrenamiento.

⚠ ADVERTENCIA 3: Tasas de aprendizaje iguales para μ y σ

Los comentarios del código mencionan que lr_mu y lr_sigma deberían ser 0.00001, pero están fijados a 0.005. Los gradientes de μ y σ incluyen factores polinómicos (z^3 , z^4) que pueden ser mucho mayores que 1, causando pasos de gradiente grandes y potencial inestabilidad numérica. Usar tasas más pequeñas para μ y σ sería más seguro.

⚠ ADVERTENCIA 4: sigma podría volverse negativa

No hay ninguna restricción que impida que σ tome valores negativos o cercanos a cero durante el entrenamiento. Si $\sigma \approx 0$, los gradientes explotan (hay σ^3 en el denominador). En una implementación robusta se usaría $\sigma = \exp(\log_sigma)$ o $\sigma = \text{softplus}(\text{params})$ para garantizar positividad.

⚠ ADVERTENCIA 5: No se imponen las condiciones de contorno exactamente

Ambas redes usan penalización suave para las CC (suma de cuadrados). Esto no garantiza que $u(0)=u(\pi)=0$ exactamente. Una mejora estándar sería usar una arquitectura que imponga las CC exactas, por ejemplo: $u(x) = x \cdot (\pi - x) \cdot NN(x)$, que siempre vale 0 en los extremos por construcción.

7.6 Posibles Mejoras

Aunque el código es matemáticamente correcto y didácticamente muy valioso, algunas mejoras aumentarían su robustez:

```

# 1. Inicialización aleatoria de C
C = np.random.randn(H, 1) * 0.01

# 2. Restricción de positividad para sigma
log_sigma = np.zeros((1, H)) # parámetro libre
sigma = np.exp(log_sigma)     # garantiza sigma > 0

# 3. Tasas desacopladas para KAN
lr_C = 0.005
lr_mu = 0.0001 # más pequeña, gradientes cuadráticos
lr_sigma = 0.0001 # ídem

# 4. Arquitectura con CC exactas
u_net = x * (np.pi - x) * u_pred_raw # siempre 0 en x=0 y x=n

```

7.7 Resumen Final del Análisis

ELEMENTO	ESTADO
Formulación del problema	✓ Correcta
Forward pass MLP	✓ Correcto
Cálculo analítico de u_{xx} MLP	✓ Correcto
Gradientes $\partial L_{PDE}/\partial(W,b,V)$	✓ Correctos (verificados)
Gradientes $\partial L_{CC}/\partial(W,b,V)$	✓ Correctos
Forward pass KAN (gaussianas)	✓ Correcto
Cálculo analítico u_{xx} KAN	✓ Correcto
Gradientes $\partial L_{PDE}/\partial(C,\mu,\sigma)$	✓ Correctos (verificados)
Gradientes $\partial L_{CC}/\partial(C,\mu,\sigma)$	✓ Correctos
Dimensiones tensores MLP/KAN	✓ Consistentes
Nomenclatura KAN	⚠ Imprecisa (es RBF)
Inicialización $C=0$	⚠ Puede ralentizar inicio
Tasas iguales μ,σ con MLP	⚠ Riesgo de inestabilidad
sigma sin restricción positividad	⚠ Riesgo de explosión
CC impuestas suavemente	⚠ No garantía exacta
CONCLUSIÓN: Código correcto matemáticamente. Advertencias son sobre robustez, no sobre errores.	

— Fin del análisis —